

Discussion Unit: DSE Analytics

DataFrames, (Datasets,
SQL)

- DataFrame optimization; Catalyst, transform resequencing
- DataFrameReader
- DataFrame creation
- DataFrame helpers
- DataFrame API; transforms, actions, expressions
- CassandraSQLConnect API
- Predicate pushdown
- (Examples)

Discussion Lab:

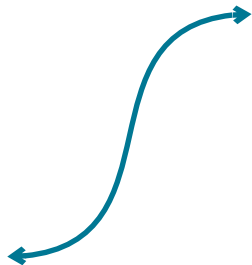
Matching pairs -- Match the attributes on the right with the areas on the left

And-

Discuss – SQL/CQL queries

DSE Analytics; DataFrames transforms and actions, (SQL), other

DSE Analytics: T | F



☐ A repartition (shuffle) will always write to disk first

———— DF in, RDD out ? ————



☐ `cache(), persist()`

☐ `map(), flatMap()`

☐ `collect()`

☐ `repartition()`

☐ `first(), take(), head()`

☐ `count()`

```
SELECT
  t1.customer_num, t1.state,
  t2.order_num, t2.paid_date
FROM
  ks_7579.customer t1, ks_7579.orders t2
```

WHERE

```
t1.customer_num =
t2.customer_num
```

AND

```
t2.customer_num = 115;
```

```
SELECT
  customer_num, company
FROM
  ks_7579.customer2
```

WHERE

```
customer_num = 104
```

```
ORDER BY company;
```

SQL/CQL Queries: How, and what index is best ?

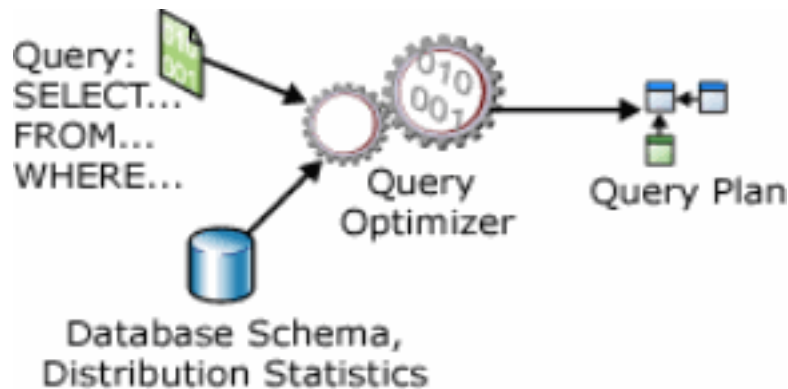


Table access method
Table order
Index negation



End of Discussion Lab:

From: Maury_Atwater
To: DSE_HOTSHOT

Subject: Need this now !!!

Big deal. You previously loaded customer_orders. ☹

Now join the SQL modeled customer, orders, and items,
and load the DSE modeled customer_orders2. We need
order_total, calculated from items (quantity * price).

I also need Jobs that show:

- . Predicate pushdown
- . Optimization

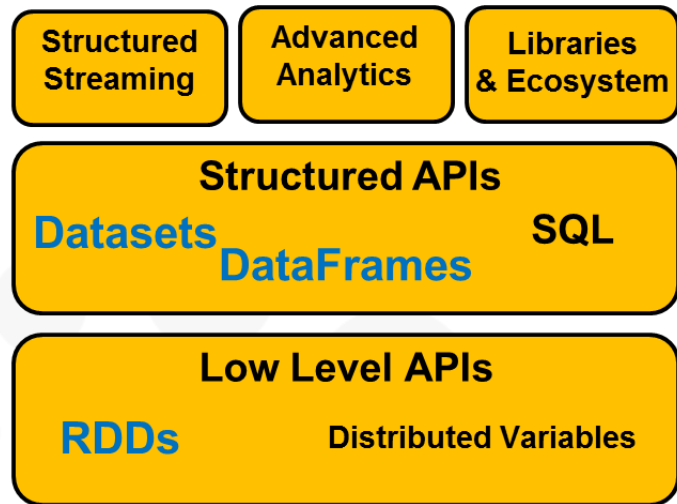
-MA

Working Goal:



Maury Atwater, President
of Atwater's

DSE Analytics: DataFrames, Datasets, (SQL)



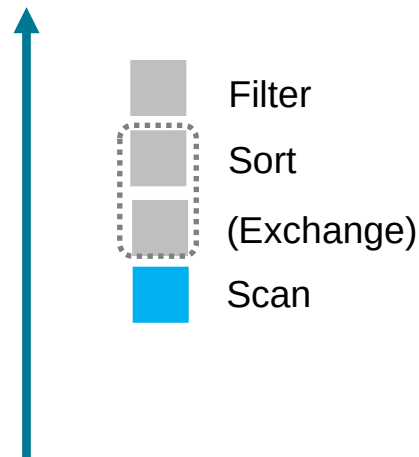
- RDD; first (core) abstraction, opaque, 2011
- Dataset; named columns/types, compile time checking, JVM platforms only, 2015
- DataFrame; fastest, runtime checking of columns/types, now alias for Dataset[Row], 2013

Both-

- Optimized execution plans via Catalyst Optimizer
- Custom memory management via Project Tungsten

Example Optimization:

```
val recs_DF = spark.  
  sql("SELECT customer_num , company " +  
    "FROM ks_7579.customer")  
  
recs_DF.explain()  
  
// == Physical Plan ==  
// *Scan org.apache.spark.sql.cassandra.CassandraSourceRelation  
//   ks_7579.customer[customer_num#28,company#32]  
//   ReadSchema: struct<customer_num:int,company:string>
```

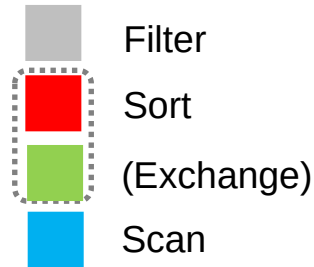


Example Optimization:

```
val recs_DF = spark.  
  sql("SELECT customer_num , company " +  
    "FROM ks_7579.customer").  
  orderBy("company")
```

```
recs_DF.explain()
```

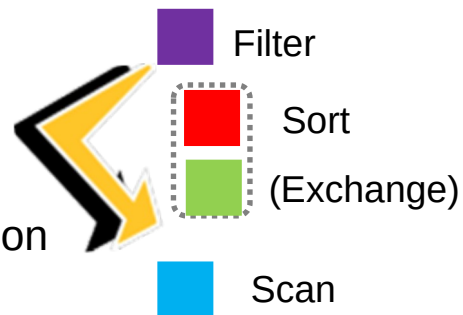
```
// == Physical Plan ==  
// *Sort [company#85 ASC NULLS FIRST], true, 0  
// +- Exchange rangepartitioning(company#85 ASC NULLS FIRST, 200)  
//   +- *Scan org.apache.spark.sql.cassandra.CassandraSourceRelation  
//     ks_7579.customer[customer_num#81,company#85] ReadSchema:  
//     struct<customer_num:int,company:string>
```



```
val recs_DF2 = recs_DF.  
  where("company > 'M'")
```

```
recs_DF2.explain()  
// == Physical Plan ==  
// *Sort [company#85 ASC NULLS FIRST], true, 0  
// +- Exchange rangepartitioning(company#85 ASC NULLS FIRST, 200)  
//   +- *Filter (company#85 > M)  
//     +- *Scan org.apache.spark.sql.cassandra.CassandraSourceRelation  
//       ks_7579.customer[customer_num#81,company#85]  
//       PushedFilters: [*IsNotNull(company), GreaterThan(company,M)],  
//       ReadSchema: struct<customer_num:int,company:string>
```

Example Optimization:



DataFrames: Creation



DataFrames: How to Create, Reflection

```
val recs = sc.parallelize(Array(
  (111, "Bob, Mary"),
  (222, "Ted"),
  (333, "Alice, Bob, Harold"),
  (444, "Dave, Bob")
))
recs.getClass
// res: Class[_ <: org.apache.spark.rdd.RDD[(Int, String)]] = class org.apache.spark.rdd.ParallelCollectionRDD

val recs_DF = recs.toDF
// recs_DF: org.apache.spark.sql.DataFrame = [_1: int, _2: string]

recs_DF.getClass
// res: Class[_ <: org.apache.spark.sql.DataFrame] = class org.apache.spark.sql.Dataset

recs_DF.printSchema()
// root
// |-- _1: integer (nullable = false)
// |-- _2: string (nullable = true)
```



Where have we seen
ordinals before ?

DataFrames: How to Create, Reflection

```
case class My_Record ( pk: Int, value: String)

val recs = sc.parallelize(Array(
  My_Record(111, "Bob, Mary"),
  ...
  My_Record(444, "Dave, Bob")
))
recs.getClass
// res: Class[_ <: org.apache.spark.rdd.RDD[My_Record]] = class org.apache.spark.rdd.ParallelCollectionRDD

val recs_DF = recs.toDF ()
// recs_DF: org.apache.spark.sql.DataFrame = [pk: int, value: string]

recs_DF.getClass
// res: Class[_ <: org.apache.spark.sql.DataFrame] = class org.apache.spark.sql.Dataset

recs_DF.printSchema()
// root
// |-- pk: integer (nullable = false)
// |-- value: string (nullable = true)
```

DataFrames: How to Create, Reflection

```
case class My_Record ( pk: Int, value: String)
```

```
val recs = sc.parallelize(Array(  
  My_Record(111, "Bob, Mary"),  
  ...  
  My_Record(444, "Dave, Bob")  
))
```

```
val recs_DF = recs.toDF ("pk", "value")
```

```
recs_DF: org.apache.spark.sql.DataFrame = [pk: int, value: string]
```

```
recs_DF.getClass
```

```
// res: Class[_ <: org.apache.spark.sql.DataFrame] = class org.apache.spark.sql.Dataset
```

```
recs_DF.printSchema()
```

```
// root
```

```
// |-- pk: integer (nullable = false)
```

```
// |-- value: string (nullable = true)
```

From the Earlier RDD Example (RDD)

```
val recs = sc.textFile("file:///opt/stores_db/7542_HelloWorld.csv")
```

```
recs.getClass()
```

```
// res: Class[_ <: org.apache.spark.rdd.RDD[String]] = class org.apache.spark.rdd.MapPartitionsRDD
```

```
recs.printSchema()
```

```
// <console>: error: value printSchema is not a member of org.apache.spark.rdd.RDD[String]
```

```
//   recs.printSchema()
```

```
// Contents of CSV file,
```

```
// 111, Bob, Mary
```

```
// 222, Ted
```

```
// 333, Alice, Bob, Harold
```

```
// 444, Dave, Bob
```

From the Earlier RDD Example (Data)

```
val recs_asDF = spark.read.format("csv").  
  option("header", "false").  
  load("file:///opt/stores_db/zzz.csv")
```

```
recs_asDF.collect().foreach(println)
```

// Note: the output below is not correct. Schema was inferred, and is wrong.

```
// [111, Bob, Mary]
```

```
// [222, Ted,null]
```

```
// [333, Alice, Bob]      Harold ?
```

```
// [444, Dave, Bob]
```

```
recs_asDF.getClass()
```

```
// res21: Class[<: org.apache.spark.sql.DataFrame] = class org.apache.spark.sql.Dataset
```

```
recs_asDF.printSchema()
```

```
// root
```

```
// |-- _c0: string (nullable = true)
```

```
// |-- _c1: string (nullable = true)
```

```
// |-- _c2: string (nullable = true)
```

Harold ?



DataFrameReader did not
like the original file name-



Where's Harold ?

DataFrames: Works with JSON

```
case class My_Record (pk: String, value: Array[String] )
```

```
// ** Notice JSON file has col as int, while case class defined it as string
```

```
// These 2 lines are the same
```

```
val recs_asDF = spark.read.format("json").load("file:///opt/stores_db/zzz.json").as[My_Record]
```

```
val recs_asDF = spark.read.json("file:///opt/stores_db/zzz.json").as[My_Record]
```

```
// recs_asDF: org.apache.spark.sql.Dataset[My_Record] = [pk: bigint, value: array<string>]
```

```
recs_asDF.collect().foreach(println)
```

```
// My_Record(111,[Ljava.lang.String;@49ae0a5)
```

```
// My_Record(222,[Ljava.lang.String;@11ac2720)
```

```
// My_Record(333,[Ljava.lang.String;@498623f0)
```

```
// My_Record(444,[Ljava.lang.String;@eaf0124)
```



println() does not provide
an overloaded method for
Array[]

So, use .show() next page

```
recs_asDF.show()
```

```
+---+-----+
| pk|          value|
+---+-----+
|111|    [Bob, Mary]|
|222|          [Ted]|
|333|[Alice, Bob, Harold]|
|444|    [Dave, Bob]|
+---+-----+
```

```
recs_asDF.show(2)
```

```
+---+-----+
| pk|      value|
+---+-----+
|111|[Bob, Mary]|
|222|      [Ted]|
+---+-----+
```

only showing top 2 rows

DataFrames: How to Create, Reflection

DataFrames: How to Create, Reflection

```
recs_asDF.getClass()  
// res: Class[_ <: org.apache.spark.sql.Dataset[My_Record]] = class org.apache.spark.sql.Dataset  
  
recs_asDF.printSchema()  
// root  
// |-- pk: long (nullable = true)  
// |-- value: array (nullable = true)  
// |   |-- element: string (containsNull = true)
```

DataFrames: How to Create, Programmatically

```
import org.apache.spark.sql.Row
import org.apache.spark.sql.types._
```

```
val recs = sc.parallelize(Array(
  (111, "Bob, Mary"),
  (222, "Ted"),
  (333, "Alice, Bob, Harold"),
  (444, "Dave, Bob")
)).
  map{case (pk, value) => Row(pk, value) }
// recs: org.apache.spark.rdd.RDD[org.apache.spark.sql.Row] = MapPartitionsRDD[3] at map
```

```
recs.printschema()
// <console>: error: value printschema is not a member of org.apache.spark.rdd.RDD[String]
//   recs.printschema()
```



These will not be in the
Scala REPL

DataFrames: How to Create, Programmatically

```
val My_Schema = StructType( List(  
  StructField("pk", IntegerType, true), // t|f nullable  
  StructField("value", StringType, true)  
) )  
// My_Schema: org.apache.spark.sql.types.StructType =  
// StructType(StructField(pk,IntegerType,true), StructField(value,StringType,true))
```

```
import spark.implicits._
```

```
val recs_DF = spark.createDataFrame(recs, My_Schema)  
// recs_DF: org.apache.spark.sql.DataFrame = [pk: int, value: string]
```

```
recs_DF.getClass  
// res: Class[_ <: org.apache.spark.sql.DataFrame] = class org.apache.spark.sql.Dataset
```

```
recs_DF.printSchema()  
// root  
// |-- pk: integer (nullable = true)  
// |-- value: string (nullable = true)
```



This will not be
in the Scala
REPL

DataFrames: How to Create, Programmatically

```
val recs_DF = spark.sql("SELECT customer_num, company FROM ks_7579.customer")
// recs_DF: org.apache.spark.sql.DataFrame = [customer_num: int, company: string]

recs_DF.getClass
// res: Class[_ <: org.apache.spark.sql.DataFrame] = class org.apache.spark.sql.Dataset

recs_DF.printSchema()
// root
// |-- customer_num: integer (nullable = true)
// |-- company: string (nullable = true)
```

DataFrames: How to Create, DataFrameReader

```
val recs_DF = spark.read.  
  format("org.apache.spark.sql.cassandra").  
  options(Map("keyspace" -> "ks_7579", "table" -> "customer")).  
  load  
  
recs_DF.getClass  
// res: Class[_ <: org.apache.spark.sql.DataFrame] = class org.apache.spark.sql.Dataset  
  
// Human readable  
recs_DF.printSchema()  
// root  
// |-- customer_num: integer (nullable = false)  
// |-- address1: string (nullable = true)  
// |-- address2: string (nullable = true)
```

```
recs_DF.printSchema()  
// root  
// |-- customer_num: integer (nullable = false)  
// |-- address1: string (nullable = true)  
// ...  
  
recs_DF.dtypes  
res: Array[(String, String)] = Array(  
  (customer_num,IntegerType),  
  (address1,StringType),  
  ...  
  
recs_DF.schema  
res: org.apache.spark.sql.types.StructType =  
  StructType(StructField(customer_num,IntegerType,false),  
    StructField(address1,StringType,true),  
    ...  
  
recs_DF.columns  
res: Array[String] = Array(customer_num, address1, ...
```

DataFrames: Helpers



Source: The Simpsons

DataFrames: Helpers

```
val recs_DF = spark.  
  sql("SELECT customer_num, company FROM ks_7579.customer").  
  first()  
// recs_DF: org.apache.spark.sql.Row = [103,Phils Sports]
```

```
println(recs_DF(0))  
println(recs_DF.getInt(0))  
103  
  
println(recs_DF.isNullAt(1))  
false
```



These are called,
(DataFrame) primitives

DataFrames: API



DataFrame: Transforms, Repartitioning, Persistence

Transform	Description
repartition(n)	Full reshuffle of the data, guaranteed balance across nodes
coalesce(n)	Above; no data movement, only change count of partitions
persist(sl)	MEMORY_ONLY, DISK_ONLY ..
cache()	Same as: persist(MEMORY_ONLY)
unpersist()	Clears (RDD, DF) from persistence list, equal to NONE

DataFrame: Transforms, DataFrame to RDD

Transform	Description
map(f)	Same as RDD, outputs RDD
flatMap(f)	Same as RDD; outputs RDD
rdd	Casts a DataFrame/Dataset to RDD, giving access to RDD only transforms
toJSON	Converts the given data/schema to same data, JSON format/schema

DataFrame: Actions

Transform	Description
collect()	Returns an array of all rows in source DataFrame
count()	Returns an int/bigint
first(), head()	Returns the first row
take(n), head(n)	Returns an array with (n) rows
show()	Displays first n rows, 20 default, strings more than 20L are truncated, human readable format

DataFrame: Actions

Transform	Description
foreach()	Executes f() on each row in source DataFrame.

DataFrame: Query API, Unary Transforms

Transform	Description
select(c1, c2, ..)	Subset (or all) columns to return; accept * and AS
withColumnRenamed(oc, nc)	Ability to rename column; oc=old-column, nc=new-column
distinct()	Eliminating duplicate rows by comparing all column values
where(p) filter(p)	where/filter, synonyms; a column expression/predicate
agg(expr)	Apply aggregate functions to one or more columns; very similar to SQL

Also know as: "Language-integrated queries"

DataFrame: Query API, Unary Transforms

Transform	Description
groupBy(c1, ..)	Similar to SQL; required when returning non-agg column (just like SQL)
orderBy(c1, ..) sort(c1, ..)	orderBy/sort, synonyms;
limit(n)	Return the first (n) rows

DataFrame: Query API, Binary Transforms

Transform	Description
join(oDF [,p], [t])	Similar to a SQL join, inner/outer joins (t), p/join-predicates
unionAll(oDF)	Similar to a SQL UNION-ALL (no duplicate remove)
intersect(oDF	Rows common to both DataFrames; union compatible
except(oDF)	Rows in source, not in oDF (other DataFrame)

DataFrame: Expressions

Transform	Description
Aggregate	avg, count, max, min, sum
Collection	array_contains, sort_array, size, ..
Date-time	current_date, current_timestamp, second, hour, month, year, ..
Math	ceil, floor, round, pow, sqrt, log, sum, sin, cos, tan
Sorting	asc, desc

```
import org.apache.spark.sql.functions._
```

DataFrame: Expressions

Transform	Description
String	concat, length, substring, trim, ..
UDF	udf, callUDF
Windowing	denseRank, percentRank, rank, lag, lean, ..
Miscellaneous	col, column, rand, randn, ..

```
import org.apache.spark.sql.functions._
```

CassandraSQLContext API-

Method	Description
<code>setKeyspace(k)</code>	Sets a default keyspace for any SQL/CQL queries
<code>sql()</code> <code>cassandraSql()</code>	Method to Spark/SQL. (<code>cassandraSQL()</code> , deprecated.)
<code>registerTempTable(df)</code>	Registers source DataFrame as a temp table (df) that can have SQL executed against it; no keyspace reference
<code>explain()</code>	Output the query plan in human readable format

DataFrame, CassandraSQLContext API Examples-

DataFrame, CassandraSQLContext:

```
val recs_DF = spark.read.  
  format("org.apache.spark.sql.cassandra").  
  options(Map("keyspace" -> "ks_7579",  
    "table" -> "customer")).  
  load  
// recs_DF: org.apache.spark.sql.DataFrame = [  
  customer_num: int, address1: string ... 8 more fields]  
  
recs_DF.show()  
+-----+-----+-----+  
|customer_num|address1|address2|...  
+-----+-----+-----+  
|103|654 Poplar|P. O. Box 3498|  
|114|947 Waverly Place|null|  
|110|520 Topaz Way|null|  
...  
only showing top 20 rows
```

DataFrame, CassandraSQLContext: column expression

```
val recs_DF = spark.  
  sql("SELECT customer_num + 5, upper(company) " +  
    "FROM ks_7579.customer")
```

```
recs_DF.show
```

```
+-----+-----+  
| (customer_num + 5) | upper(company) |  
+-----+-----+  
|          108 | PHILS SPORTS |  
|          119 | SPORTING PLACE |  
|          115 | AA ATHLETICS |  
|          ... |               |
```



Use case ?

```
import org.apache.spark.sql._           // Row
import org.apache.spark.sql.types._     // StructType
import org.apache.spark.sql.functions.lit
import org.apache.spark.sql.functions._
```

```
val recs_DF = spark.
  sql("SELECT customer_num, company " +
    "FROM ks_7579.customer")
```

```
val recs_DF2 = recs_DF.withColumn("col8", functions.lit("Another String"))
recs_DF2.show
```

customer_num	company	col8
103	Phils Sports	Another String
114	Sporting Place	Another String
110	AA Athletics	Another String
128	Phoenix University	Another String
117	Kids Korner	Another String
120	Century Pro Shop	Another String

DataFrame, CassandraSQL Context: column expression



Use case ?


```
import org.apache.spark.sql._           // Row
import org.apache.spark.sql.types._     // StructType
import org.apache.spark.sql.functions.lit
import org.apache.spark.sql.functions._
```

```
val my_upper: (String => String) = (arg: String) =>
    { arg.toUpperCase }
// my_upper: String => String = <function1>
```

```
val sqlfunc = udf(my_upper)
// sqlfunc: org.apache.spark.sql.expressions.UserDefinedFunction =
UserDefinedFunction(<function1>,StringType,Some(List(StringType)))
```

```
val recs_DF3 = recs_DF2.withColumn("col2", sqlfunc(col("company")))
recs_DF3.show
```

customer_num	company	col8	col2
103	Phils Sports	Another String	PHILS SPORTS
114	Sporting Place	Another String	SPORTING PLACE
110	AA Athletics	Another String	AA ATHLETICS
128	Phoenix University	Another String	PHOENIX UNIVERSITY

DataFrame,
CassandraSQLContext:
column expression



udf() use case ?

DataFrame, CassandraSQLContext: RDD cast

```
import org.apache.spark.sql._           // Row
import org.apache.spark.sql.types._     // StructType
import org.apache.spark.sql.functions.lit
import org.apache.spark.sql.functions._
```

```
val recs_RDD = spark.
  sql("SELECT customer_num + 5, upper(company) " +
    "FROM ks_7579.customer").rdd
```



Use case ?

DataFrame, CassandraSQLContext: Aggregation

```
import org.apache.spark.sql.functions._

recs_DF.
  filter("customer_num > 102").
  groupBy("state").
  agg(Map("*" -> "count")).
  withColumnRenamed("COUNT(1)", "state_count").
  select("state", "state_count").
  orderBy(desc("state_count")).
  limit(5).
  show
```

```
+-----+-----+
|state|state_count|
+-----+-----+
|  CA |         16|
|  NJ |          2|
|  AZ |          2|
|  NY |          1|
|  MA |          1|
+-----+-----+
```

DataFrame, CassandraSQLContext: Aggregation

```
val recs_DF = spark.  
  sql("SELECT company, MAX(zipcode) AS max_zipcode "  
    "FROM ks_7579.customer GROUP BY company").  
  show
```

```
+-----+-----+  
|          company|max_zipcode|  
+-----+-----+  
|      Bay Sports|      32256|  
|    Kids Korner|      94063|
```

...

only showing top 20 rows

```
val recs_DF = spark.  
  sql("SELECT company, MAX(zipcode) AS max_zipcode "  
    "FROM ks_7579.customer GROUP BY company")  
  ...
```

DataFrame, CassandraSQLContext: Aggregation

```
...  
val recs_DF = spark.  
  sql("SELECT company, MAX(zipcode) AS max_zipcode " +  
    "FROM ks_7579.customer GROUP BY company")
```



Why do some stages
repeat ?

```
recs_DF.explain  
== Physical Plan ==  
SortAggregate(key=[company#1319], functions=[max(zipcode#1324)])  
+- *Sort [company#1319 ASC NULLS FIRST], false, 0  
  +- Exchange hashpartitioning(company#1319, 200)  
    +- SortAggregate(key=[company#1319], functions=[partial_max(zipcode#1324)])  
      +- *Sort [company#1319 ASC NULLS FIRST], false, 0  
        +- *Scan org.apache.spark.sql.cassandra.CassandraSourceRelation  
          ks_7579.customer[company#1319,zipcode#1324]  
          ReadSchema: struct<company:string,zipcode:string>
```

DataFrame, CassandraSQLContext: Writing

```
val recs_DF = spark.read.  
  format("org.apache.spark.sql.cassandra").  
  options(Map("keyspace" -> "ks_7579", "table" -> "customer2")).  
  load.  
  count()  
// recs_DF: Long = 0  
  
// DataFrameReader  
val recs_DF = spark.read.  
  format("org.apache.spark.sql.cassandra").  
  options(Map("keyspace" -> "ks_7579", "table" -> "customer")).  
  load  
// recs_DF: org.apache.spark.sql.DataFrame = [customer_num: int, address1: string ... 8 more  
fields]  
  
...
```

DataFrame, CassandraSQLContext: Writing

```
...  
  
recs_DF.write.  
  format("org.apache.spark.sql.cassandra").  
  options(Map( "keyspace" -> "ks_7579", "table" -> "customer2" )).  
  save  
// no output;  
  
val recs_DF = spark.read.  
  format("org.apache.spark.sql.cassandra").  
  options(Map("keyspace" -> "ks_7579", "table" -> "customer2")).  
  load.  
  count()  
// recs_DF: Long = 28
```

DataFrame, CassandraSQLContext: Transform Before Write

```
// cqlsh: truncate customer2;
```

```
val recs_DF = spark.read.  
  format("org.apache.spark.sql.cassandra").  
  options(Map("keyspace" -> "ks_7579", "table" -> "customer2")).  
  load.  
  count()
```

```
// recs_DF: Long = 0
```

```
val recs_DF = spark.read.  
  format("org.apache.spark.sql.cassandra").  
  options(Map("keyspace" -> "ks_7579", "table" -> "customer")).  
  load.
```

```
  filter($"fname" === "Frank" || $"lname" === "Miller")
```

```
// recs_DF: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [customer_num: int,  
address1: string ... 8 more fields]
```

```
...
```


DataFrame, CassandraSQLContext: Transform Before Write

```
...  
  
recs_DF.write.  
  format("org.apache.spark.sql.cassandra").  
  options(Map( "keyspace" -> "ks_7579", "table" -> "customer2" )).  
  save  
// no output;  
  
val recs_DF = spark.read.  
  format("org.apache.spark.sql.cassandra").  
  options(Map("keyspace" -> "ks_7579", "table" -> "customer2")).  
  load.  
  count()  
// recs_DF: Long = 3
```

DataFrame, CassandraSQLContext: write mode

```
val recs_DF = spark.  
  sql("SELECT customer_num, company FROM ks_7579.customer").  
  filter($"fname" === "Frank" || $"lname" === "Miller")  
  
recs_DF.write.  
  format("org.apache.spark.sql.cassandra").  
  options(Map( "keyspace" -> "ks_7579", "table" -> "customer2" )).  
  save  
// java.lang.UnsupportedOperationException: 'SaveMode is set to ErrorIfExists and Table  
// ks_7579.customer2 already exists and contains data.  
// Perhaps you meant to set the DataFrame write mode to Append?
```

DataFrame, CassandraSQLContext: write mode

```
// Write with proper Append mode-

val recs_DF = spark.
  sql("SELECT customer_num, company FROM ks_7579.customer").
  filter($"fname" === "Frank" || $"lname" === "Miller")

// append, overwrite, other,
// See,
// https://spark.apache.org/docs/2.2.0/api/java/org/apache/spark/sql/DataFrameWriter.html
recs_DF.write.
  mode("append").
  format("org.apache.spark.sql.cassandra").
  options(Map( "keyspace" -> "ks_7579", "table" -> "customer2" )).
```

DSE Analytics: Predicate Pushdown

Any predicate that is valid in CQL is pushed from (Spark) down to DSE for processing when ..

- Supported by a partition key
- Support by a clustering key

Predicate Pushdown: Example

```
val recs_DF = spark.  
  sql("SELECT customer_num , company " +  
    "FROM ks_7579.customer").  
  orderBy("company")  
  
val recs_DF2 = recs_DF.  
  where("customer_num > 115")
```

```
recs_DF2.explain()  
// == Physical Plan ==  
// *Sort [company#106 ASC NULLS FIRST], true, 0  
// +- Exchange rangepartitioning(company#106 ASC NULLS FIRST, 200)  
//    +- *Filter (customer_num#102 > 115)  
//       +- *Scan org.apache.spark.sql.cassandra.CassandraSourceRelation  
ks_7579.customer[customer_num#102,company#106] PushedFilters: [*NotNull(customer_num),  
GreaterThan(customer_num,115)], ReadSchema: struct<customer_num:int,company:string>
```



Filter



Scan



(Exchange)

Sort

Predicate Pushdown: Example



Filter



Scan



(Exchange)

Sort

```
val recs_DF2 = recs_DF.
```

```
  where("customer_num = 115")
```

```
recs_DF2.explain()
```

```
// == Physical Plan ==
```

```
// *Sort [company#106 ASC NULLS FIRST], true, 0
```

```
// +- Exchange rangepartitioning(company#106 ASC NULLS FIRST, 200)
```

```
//   +- *Scan org.apache.spark.sql.cassandra.CassandraSourceRelation
```

```
ks_7579.customer[customer_num#102,company#106] PushedFilters: [*NotNull(customer_num),  
*EqualTo(customer_num,115)], ReadSchema: struct<customer_num:int,company:string>
```



End of Unit:



Additional Content:

Limited API Allows Catalyst

Catalyst is an Optimizer which can Re-plan User requests

```
ds.groupBy('value').count().explain(true)
== Parsed Logical Plan ==
'Aggregate ['value], [unresolvedalias('value, None), count(1) AS count#47L]
+- LocalRelation [value#22]

== Analyzed Logical Plan ==
value: string, count: bigint
Aggregate [value#22], [value#22, count(1) AS count#47L]
+- LocalRelation [value#22]

== Optimized Logical Plan ==
Aggregate [value#22], [value#22, count(1) AS count#47L]
+- LocalRelation [value#22]

== Physical Plan ==
*HashAggregate(keys=[value#22], functions=[count(1)], output=[value#22, count#47L])
+- Exchange hashpartitioning(value#22, 200)
   +- *HashAggregate(keys=[value#22], functions=[partial_count(1)], output=[value#22, count#53L])
      +- LocalTableScan [value#22]
```

Parsed From API

Find all Attributes,
Expand Wildcards

Reorganize Plan

Actually Figure
Out Operations